

AI Conversation & Sales Suite

Detailed Product Requirements Document (PRD)

A complete beginner-friendly blueprint covering product requirements, AI agents, architecture, database, RAG, integrations, security, testing, deployment, and implementation phases.

Recommended stack: Next.js + React + TypeScript | Laravel + PHP | MySQL | Redis | Qdrant | Docker | AI APIs / Local LLM

Version 1.0 • September 2026

Table of Contents

- 1. Executive Summary
- 2. Problem Statement
- 3. Product Vision & Goals
- 4. Users & Roles
- 5. Product Scope
- 6. End-to-End User Journey
- 7. System Hierarchy
- 8. Functional Requirements
- 9. Conversation Engine
- 10. AI Orchestrator
- 11. Customer Support Agent
- 12. Sales Agent
- 13. Appointment Agent
- 14. Human Handoff
- 15. Knowledge Base & RAG
- 16. Tool Calling
- 17. CRM & Lead Management
- 18. Calendar & Appointments
- 19. E-commerce & Business Integrations
- 20. Notifications
- 21. Analytics & Usage
- 22. Multi-Tenancy & RBAC
- 23. Database / MySQL
- 24. Vector Database
- 25. Complete Tech Stack
- 26. System Architecture
- 27. API Architecture
- 28. Security
- 29. Reliability & Error Handling
- 30. Testing
- 31. Deployment
- 32. Free / Local Version
- 33. MVP
- 34. Development Phases
- 35. Acceptance Criteria
- 36. Example Scenarios

- 37. Future Roadmap
- 38. Final Implementation Checklist

1. Executive Summary

AI Conversation & Sales Suite is a multi-tenant SaaS platform that allows a business to communicate with website visitors and customers through AI-powered conversations. Instead of one generic chatbot, the system uses specialized agents for customer support, sales, and appointment booking.

The important architectural idea is that the agents share the same core conversation system, customer profile, knowledge base, tools, integrations, analytics, and human handoff layer. The AI decides which capability is required, while the backend controls what actions are actually allowed.

Core principle: the LLM should not directly modify the database or external systems. It requests a typed tool/action; Laravel validates authorization, tenant ownership, input, business rules, and integration credentials before executing it.

Primary channels for MVP: website chat first. Messaging channels such as WhatsApp, Instagram, SMS, or other providers can be added through channel adapters later.

Recommended MVP: multi-tenant dashboard + web chat + AI orchestrator + knowledge base/RAG + sales qualification + lead creation + Google Calendar + human handoff + basic analytics.

2. Problem Statement

Businesses receive repetitive questions, lose leads outside working hours, spend staff time qualifying prospects, and manually schedule appointments. A normal FAQ chatbot solves only a small part of this problem.

The product should close the loop from conversation to action: answer questions, understand intent, qualify the visitor, capture lead information, schedule an appointment, update CRM data, and hand off to a human when AI should stop.

Business problems addressed:

- Slow or inconsistent customer support.
- Leads lost because nobody responds immediately.
- Sales teams spend time on low-value qualification.
- Appointment booking requires back-and-forth messages.
- Business knowledge is scattered across documents and websites.
- Managers cannot easily see AI conversations, outcomes, usage, and failures.

3. Product Vision & Goals

Vision: provide an AI operations layer that can communicate naturally with customers and safely perform useful business actions.

Goals:

- 24/7 customer communication.
- Improve lead capture and qualification.
- Increase appointment conversion.
- Reduce repetitive support workload.
- Ground answers in approved business knowledge.
- Give staff visibility and control.
- Keep each organization's data strictly isolated.

- Make agents and integrations modular so new capabilities can be added without rewriting the platform.

Non-goals for the first release: building a foundation model, replacing every CRM, fully autonomous financial transactions, unrestricted browser automation, or letting AI perform arbitrary database/API operations.

4. Users & Roles

Role	Main responsibilities
Platform Super Admin	Manage organizations, system settings, providers, platform health, plans, and global audit information.
Organization Owner/Admin	Configure business, AI agents, knowledge base, integrations, team members, and permissions.
Manager	Review conversations, leads, appointments, analytics, and agent performance.
Sales/Support Staff	View assigned conversations, take over chats, update leads, and follow up.
Agent/Operator	Handle specific operational conversations or appointments according to permissions.
End Customer/Visitor	Chat with the AI, ask questions, submit details, book/reschedule appointments, and request human help.

The application must use tenant-aware authorization. A user belonging to Organization A must never access Organization B's customers, conversations, documents, integrations, or analytics.

5. Product Scope

Core modules:

- Authentication and organization management.
- Tenant-aware dashboard.
- Conversation inbox and chat UI.
- AI orchestration and agent routing.
- Support, Sales, and Appointment agents.
- Knowledge base and RAG.
- Tool/action layer.
- Lead and customer management.
- Calendar and appointment management.
- Human handoff.
- Notifications.
- Analytics and usage/token tracking.
- Integration management.
- Security, audit logs, and administration.

6. End-to-End User Journey

Example: a visitor opens a company's website at 11:30 PM.

```

Visitor → Website Chat Widget → Conversation API → AI Orchestrator
→ Intent Detection → Sales Agent
→ Knowledge Retrieval + Business Tools
→ Qualification → Lead Creation
→ Appointment Agent (if requested)
→ Calendar Availability → Booking
→ Confirmation → Analytics

```

At every step, the system should store the conversation state and important business events. The AI response is generated from approved context, but external actions are executed by backend tools.

7. System Hierarchy

```
PLATFORM
■ ■ ■ ■ Organizations / Tenants
■ ■ ■ ■ Users & Roles
■ ■ ■ ■ AI Configuration
■ ■ ■ ■ Knowledge Base
■ ■ ■ ■ Customers / Contacts
■ ■ ■ ■ Leads
■ ■ ■ ■ Conversations
■ ■ ■ ■ Appointments
■ ■ ■ ■ Integrations
■ ■ ■ ■ Analytics / Usage
■
■ ■ ■ ■ Conversation Layer
■ ■ ■ ■ Channel Adapter
■ ■ ■ ■ Conversation API
■ ■ ■ ■ Message Store
■ ■ ■ ■ Context Builder
■ ■ ■ ■ Human Handoff
■
■ ■ ■ ■ AI Layer
■ ■ ■ ■ AI Orchestrator
■ ■ ■ ■ Support Agent
■ ■ ■ ■ Sales Agent
■ ■ ■ ■ Appointment Agent
■ ■ ■ ■ Prompt Manager
■ ■ ■ ■ Guardrails
■
■ ■ ■ ■ Knowledge Layer
■ ■ ■ ■ File Upload
■ ■ ■ ■ Text Extraction
■ ■ ■ ■ Chunking
■ ■ ■ ■ Embeddings
■ ■ ■ ■ Qdrant
■ ■ ■ ■ Retrieval / Reranking
■
■ ■ ■ ■ Tool Layer
■ ■ ■ ■ CRM Tools
■ ■ ■ ■ Calendar Tools
■ ■ ■ ■ Customer Tools
■ ■ ■ ■ Order Tools
■ ■ ■ ■ Notification Tools
```

8. Functional Requirements

FR-01 Authentication: users can sign in, sign out, reset credentials, and belong to one or more organizations according to the product's membership model.

FR-02 Organization: an organization can configure name, business details, timezone, business hours, AI behavior, allowed channels, and integrations.

FR-03 Conversation: visitors can start a conversation, send messages, receive AI replies, and continue an existing session.

FR-04 Agent routing: the system selects Support, Sales, Appointment, or human handoff based on intent and policy.

FR-05 Knowledge: admins can upload approved documents and maintain a searchable knowledge base.

FR-06 Lead: AI can collect name/contact/need and create or update a lead through a backend tool.

FR-07 Appointment: AI can check availability and book/reschedule/cancel where the connected calendar supports it.

FR-08 Handoff: users can request a human, and the system can automatically hand off according to configured rules.

FR-09 Analytics: admins can see conversation volume, agent usage, lead outcomes, appointments, and AI usage.

FR-10 Audit: sensitive actions and configuration changes must be logged.

9. Conversation Engine

The conversation engine is the shared foundation for all agents. It should be channel-independent: the same conversation logic should work whether the message arrives from the website widget or a future external channel.

Conversation object: organization_id, customer/contact, channel, status, assigned user, active agent, metadata, timestamps.

Message object: conversation_id, sender type, content, attachments, token/usage metadata, tool calls, status, timestamps.

Conversation states: active, waiting_for_customer, waiting_for_human, resolved, archived.

The frontend should stream AI responses where practical. The backend remains the source of truth for conversation state and actions.

10. AI Orchestrator

The orchestrator is the brain that coordinates agents rather than performing every task itself.

Responsibilities: identify intent, load customer context, retrieve relevant knowledge, choose an agent, expose only allowed tools, enforce policy, manage state, and decide when to hand off.

```
Input
↓
Conversation Context
↓
Intent / Agent Routing
↓
Relevant Knowledge + Customer Data
↓
Agent Prompt + Allowed Tools
↓
LLM
↓
Response OR Tool Request
↓
Laravel Tool Validation
↓
External Action / Database Update
↓
Final Response + Audit Event
```

The orchestrator should use structured outputs where possible so routing and tool requests are machine-readable rather than inferred from free-form text.

11. Customer Support Agent

Purpose: resolve routine support questions quickly and safely.

- Answer FAQs from approved knowledge.
- Explain policies, services, hours, and basic troubleshooting.
- Retrieve supported customer/order information through tools.
- Create a support request or handoff when needed.

- Avoid inventing policies or unsupported facts.

Escalation triggers: user explicitly asks for a human; issue requires privileged access; knowledge confidence is low; repeated failure; sensitive complaint; or business-defined escalation condition.

12. Sales Agent

Purpose: turn conversations into qualified opportunities without using manipulative or deceptive tactics.

- Understand customer needs.
- Answer product/service questions.
- Recommend relevant options using approved catalog/business data.
- Capture qualification information.
- Create/update leads.
- Offer a booking or next step.
- Provide approved links or contact options.

Suggested qualification fields: name, contact method, product/service interest, use case, urgency, budget range where appropriate, location, preferred contact time, and consent/preferences.

13. Appointment Agent

Purpose: manage appointment conversations using a connected calendar or scheduling provider.

- Ask for service/type when required.
- Use organization timezone.
- Check real availability through the integration.
- Offer available slots.
- Confirm before booking.
- Create appointment and store external event ID.
- Support rescheduling/cancellation when permitted.
- Send confirmation through configured notification channels.

The AI must never claim a booking succeeded until the calendar integration confirms success.

14. Human Handoff

Human handoff is a first-class feature, not an error condition.

- User can request a human.
- Admin can define automatic escalation rules.
- Conversation status changes to `waiting_for_human`.
- Assigned staff can take over.
- AI should stop or become read-only according to the organization's handoff policy.
- The staff member can return the conversation to AI.
- Handoff and takeover events are logged.

15. Knowledge Base & RAG

RAG (Retrieval-Augmented Generation) allows the AI to answer from the organization's own approved information without training a new model.

Pipeline: upload → extract text → clean → split into chunks → generate embeddings → store vectors in Qdrant → retrieve relevant chunks → optionally rerank → place context into the agent prompt.

- Documents can include FAQs, SOPs, product information, service details, policies, and internal support material.
- Each document/chunk must carry `organization_id` and source metadata.
- Deleting a document must also remove/deactivate its vectors.
- Retrieval must be tenant-isolated.
- Live information such as current appointment availability or order status should come from tools/APIs, not stale RAG content.

Example: a document says the business is open Monday–Friday. RAG can answer the policy. For today's actual appointment slots, the Appointment Agent must call the calendar tool.

16. Tool Calling

Tools are controlled backend functions that the AI may request. They form the security boundary between the LLM and the real system.

Example tools: `search_knowledge`, `get_customer`, `create_lead`, `update_lead`, `get_order_status`, `get_calendar_availability`, `create_appointment`, `reschedule_appointment`, `cancel_appointment`, `notify_human`.

Every tool must define: name, purpose, input schema, output schema, required permission, tenant scope, validation rules, rate limit, timeout, audit behavior, and failure behavior.

```
AI requests:
create_lead({
  name: "...",
  contact: "...",
  interest: "...",
})

Laravel:
1. Validate JSON schema
2. Identify authenticated tenant
3. Check AI/tool permission
4. Validate business rules
5. Execute DB/API action
6. Write audit event
7. Return structured result
```

17. CRM & Lead Management

The platform should maintain a lightweight internal customer/lead model even when an external CRM is connected. This gives the product a consistent internal data layer.

- Contacts/customers.
- Leads and lifecycle status.
- Lead source and channel.
- Assigned owner.
- Conversation history.
- Notes and tags.

- Qualification fields.
- External CRM IDs.
- Lead events such as created, qualified, booked, converted, lost.

External CRM synchronization should be idempotent and use stored external IDs to avoid duplicate records.

18. Calendar & Appointments

Appointment data should include `organization_id`, `customer_id`, `service`, `start/end time`, `timezone`, `status`, `assigned staff/calendar`, `external provider`, `external event ID`, and `timestamps`.

- Calendar credentials are stored as encrypted secrets.
- Availability is read from the external provider.
- Booking uses provider confirmation.
- Webhook events update local state when supported.
- Timezone conversion must be explicit.
- Duplicate booking protection is required.

19. E-commerce & Business Integrations

Integrations should be implemented behind adapters/interfaces so the core product does not depend on one vendor.

Potential integrations: Google Calendar, CRMs, e-commerce platforms, email providers, SMS/WhatsApp providers, payment/checkout systems, and webhooks.

For each integration store `provider`, `organization`, `encrypted credentials`, `scopes`, `status`, `external account ID`, `last sync`, and `error state`.

Use OAuth 2.0 where the provider supports it. Verify inbound webhooks using the provider's signature mechanism and reject invalid requests.

20. Notifications

Notifications can be triggered by lead creation, appointment booking, cancellation, human handoff, failed integrations, and important system events.

- Email.
- In-app notifications.
- SMS/WhatsApp through supported providers.
- Webhook notifications.

Notifications should be queued through Redis-backed jobs so a slow provider does not block the user's conversation.

21. Analytics & Usage

Business analytics: conversations, resolved conversations, handoffs, leads, qualified leads, appointments, conversion events, response time, and agent usage.

AI analytics: model used, input/output tokens where available, estimated cost, latency, tool calls, failures, and fallback rate.

Operational analytics: queue failures, integration errors, webhook failures, API response time, and system health.

Token usage should be tracked per organization so the platform can support quotas, internal monitoring, and future billing.

22. Multi-Tenancy & RBAC

Every business is an organization/tenant. Business records must contain `organization_id` either directly or through a guaranteed tenant relationship.

Minimum roles: Owner, Admin, Manager, Agent/Staff, Viewer. Exact roles can be customized later.

- Authorization is checked server-side.
- Frontend hiding a button is not a security control.
- Integration credentials are never exposed to the browser.
- Tenant IDs should be derived from authenticated context rather than trusted from arbitrary client input.
- Background jobs must carry tenant context explicitly.

23. Database / MySQL

MySQL is the primary relational database. It stores transactional business data, configuration, users, conversations, leads, appointments, documents metadata, integration records, audit logs, and usage records.

Core tables:

Table	Purpose
organizations	Tenant/business record.
users	Platform users.
organization_user	Membership and role.
customers	End customers/contacts.
conversations	Conversation sessions.
messages	Individual messages.
agents	Agent configuration.
knowledge_documents	Uploaded document metadata.
knowledge_chunks	Text chunks and retrieval metadata.
leads	Sales opportunities.
appointments	Booking records.
integrations	Connected external providers.
integration_credentials	Encrypted credential references/data.
tool_executions	AI tool calls and results metadata.
notifications	Notification jobs/results.
usage_records	AI/API usage.
audit_logs	Security and important business events.

Important: vectors are recommended to live in Qdrant rather than forcing the relational database to become the vector search engine. MySQL remains the source of truth for business entities.

24. Vector Database / Qdrant

Qdrant is a vector database suitable for the RAG retrieval layer. It can be self-hosted, making it attractive for an affordable or local deployment.

A vector record should contain an embedding plus payload such as organization_id, document_id, chunk_id, source type, title, permissions, and status.

The application must always filter retrieval by organization_id and any applicable knowledge-base permissions.

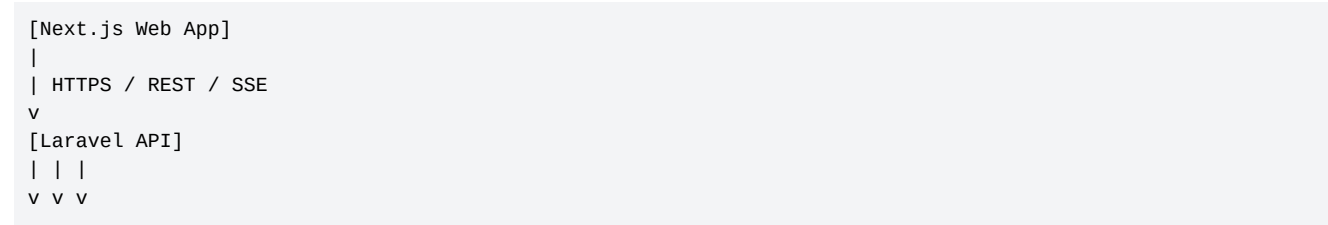
RAG flow: user question → embedding → Qdrant search with tenant filter → relevant chunks → context builder → LLM.

25. Complete Tech Stack

Layer	Technology	Role
Frontend	Next.js + React + TypeScript	Dashboard, chat UI, admin screens.
UI	Tailwind CSS + shadcn/ui	Reusable interface components.
Backend	Laravel + PHP	REST API, business logic, auth, tools.
Database	MySQL	Primary relational data.
Cache/Queue	Redis	Caching, jobs, rate limiting, temporary state.
Queue monitoring	Laravel Horizon	Monitor Redis-backed jobs.
Vector DB	Qdrant	RAG vector search.
AI	OpenAI / Gemini / Claude or local model	Language understanding/generation.
Embeddings	Selected embedding provider / local model	Knowledge retrieval.
Storage	S3-compatible storage / local storage	Documents and attachments.
Auth	Laravel Sanctum or OAuth where required	Application authentication.
API	REST + Webhooks	Frontend/integration communication.
Realtime	SSE/WebSockets	Streaming messages and live updates.
Containers	Docker	Consistent development/deployment.
Web server	Nginx	Reverse proxy / HTTP serving.
Version control	Git + GitHub	Source control and collaboration.
Monitoring	Sentry or equivalent	Errors and diagnostics.
CI/CD	GitHub Actions	Automated tests/deployments.

Cost note: most framework/open-source components are free to use. AI APIs, cloud hosting, object storage, and some third-party providers may charge based on usage. A fully local setup can replace paid AI APIs with a locally hosted model if the available hardware is sufficient.

26. System Architecture



```

MySQL Redis Qdrant
| | |
| | +--> RAG Retrieval
| +-----> Queues / Cache
+-----> Business Data
|
v
[AI Orchestrator]
| | |
v v v
Support Sales Appointment
| | |
+-----+-----+
|
Tool Layer
|
+-----+-----+
| | |
CRM Calendar Notifications
|
External APIs

```

The frontend should never directly call privileged external integrations. It communicates with Laravel; Laravel handles authorization and provider credentials.

27. API Architecture

Example endpoint groups:

```

POST /api/auth/login
GET /api/me
GET /api/organizations/{id}
GET /api/conversations
POST /api/conversations
GET /api/conversations/{id}/messages
POST /api/conversations/{id}/messages
POST /api/knowledge/documents
GET /api/leads
POST /api/leads
GET /api/appointments/availability
POST /api/appointments
POST /api/integrations/{provider}/connect
POST /api/webhooks/{provider}
GET /api/analytics/overview
GET /api/usage

```

Use request validation, API versioning, pagination, consistent error responses, rate limits, authorization policies, and idempotency for actions that may be retried.

28. Security

- HTTPS everywhere in production.
- Password hashing and secure session/token handling.
- Server-side RBAC and tenant isolation.
- Encrypted integration credentials.
- Webhook signature verification.
- Rate limiting for public chat/API endpoints.
- Input validation and output sanitization.
- Prompt-injection defenses and tool allowlists.

- Never expose system prompts or secrets.
- Audit logs for privileged configuration and external actions.
- Least-privilege OAuth scopes.
- Secure file upload validation and malware/security scanning where appropriate.
- Data retention/deletion policies.

Prompt injection defense: treat retrieved documents and user messages as untrusted data. They must not be allowed to redefine system rules or grant themselves permissions.

29. Reliability & Error Handling

- LLM timeout → controlled retry/fallback.
- Provider rate limit → exponential backoff where safe.
- Calendar failure → do not claim booking success.
- CRM failure → preserve conversation and queue synchronization.
- Qdrant failure → use a safe fallback response rather than inventing facts.
- Queue failure → retry with bounded attempts and dead-letter/error monitoring.
- Duplicate webhook → idempotency check.
- Unexpected tool result → validate before returning to the LLM.

User-facing errors should be understandable and should never expose stack traces, credentials, provider secrets, or internal infrastructure details.

30. Testing

Unit tests: business rules, validators, permission checks, parsers, tool schemas.

Feature/API tests: authentication, tenant isolation, CRUD, conversations, leads, appointments, webhooks.

AI evaluation: intent routing accuracy, factuality against approved knowledge, tool-selection accuracy, refusal/escalation behavior, prompt-injection resistance.

Integration tests: mocked and sandbox provider APIs.

End-to-end tests: visitor starts chat → qualifies → lead created → appointment booked → confirmation → analytics event.

Load tests: concurrent conversations, queue throughput, webhook bursts, database query performance.

31. Deployment

Development: Docker Compose can run Next.js, Laravel, MySQL, Redis, Qdrant, and Nginx locally.

Production baseline: Linux VPS/cloud VM, Nginx, application containers/processes, MySQL, Redis, Qdrant, object storage, TLS, backups, monitoring.

- Use separate environment variables/secrets for each environment.
- Run database migrations through CI/CD.
- Schedule backups.
- Monitor queue workers.
- Monitor application errors and provider failures.

- Use health-check endpoints.

32. Free / Local Version

For an internal company deployment, the platform can be built mostly with free/open-source software.

```
Local machine/server
■■■ Next.js
■■■ Laravel
■■■ MySQL
■■■ Redis
■■■ Qdrant
■■■ Nginx
■■■ Docker
■■■ Ollama / local LLM (optional)
```

This avoids mandatory per-request AI API charges, but local AI quality and speed depend heavily on available CPU/GPU/RAM. External integrations such as Google Calendar or messaging providers can still introduce their own limits or costs.

For development, using an external AI API can be easier and faster. For an internal production environment, a local model can be evaluated after the product workflow is stable.

33. MVP

MVP should deliberately stay focused.

- Organization + user authentication.
- Dashboard.
- Website chat widget.
- Conversation inbox.
- AI orchestrator.
- Support + Sales + Appointment agents.
- Knowledge base/RAG.
- Lead creation.
- Google Calendar integration.
- Human handoff.
- Basic analytics and token/usage tracking.
- Audit logs.

Avoid building every channel, every CRM, advanced billing, complex automation, and autonomous actions in the first release.

34. Development Phases

Phase	Area	Deliverables
Phase 1	Discovery & UX	Finalize requirements, user journeys, wireframes, roles, MVP scope.
Phase 2	Project Foundation	Create repositories, Docker, Next.js, Laravel, MySQL, Redis, Qdrant, environments.
Phase 3	Auth & Multi-Tenancy	Users, organizations, memberships, roles, policies, tenant middleware.
Phase 4	Core Conversation	Chat widget, conversations, messages, streaming, inbox.

Phase	Area	Deliverables
Phase 5	AI Core	Provider abstraction, prompts, orchestrator, structured outputs, usage tracking.
Phase 6	RAG	Upload, extraction, chunking, embeddings, Qdrant retrieval, admin KB.
Phase 7	Support Agent	FAQs, knowledge answers, escalation, support tools.
Phase 8	Sales Agent	Qualification, lead creation/update, recommendations, next-step logic.
Phase 9	Appointment Agent	Availability, booking, reschedule/cancel, confirmations.
Phase 10	Integrations	Calendar, CRM, notifications, webhooks, OAuth.
Phase 11	Human Handoff	Inbox assignment, takeover, AI pause/resume.
Phase 12	Analytics	Dashboards, funnel metrics, AI usage, tool analytics.
Phase 13	Security & QA	Security review, AI evaluation, load tests, tenant-isolation tests.
Phase 14	Production	Backups, monitoring, CI/CD, deployment, documentation.
Phase 15	V2	Additional channels, advanced automation, billing, more integrations, improved AI evaluation.

35. Acceptance Criteria

Area	Acceptance condition
Conversation	A visitor can start and continue a conversation and receive a valid AI response.
Tenant isolation	A user cannot read or modify another organization's records through UI or API.
RAG	A configured document can be indexed and relevant chunks can be retrieved for a question.
Support	Support agent answers approved FAQs and escalates when confidence/policy requires it.
Sales	Sales agent can qualify a visitor and create/update a lead.
Appointment	Appointment agent only confirms a booking after provider confirmation.
Handoff	A visitor can request a human and the conversation enters the configured handoff state.
Integrations	OAuth credentials are stored securely and provider webhooks are verified.
Audit	Privileged configuration and tool actions create audit records.
Reliability	Provider/API failures do not result in fabricated success messages.
Analytics	Conversation, lead, appointment, and AI usage events appear in the dashboard.

36. Example Scenarios

Scenario A — Support: Visitor asks a business-hours question. Orchestrator routes to Support. RAG retrieves the approved policy. Agent answers. Conversation is marked resolved if no further help is needed.

Scenario B — Sales: Visitor asks which service fits their needs. Sales Agent asks a few relevant qualification questions, retrieves approved service information, creates a lead, and offers a consultation.

Scenario C — Appointment: Visitor wants a consultation tomorrow. Appointment Agent checks real calendar availability, offers available times, receives confirmation, creates the event, stores the external event ID, and sends confirmation.

Scenario D — Human: Visitor says they want a person. The conversation changes to `waiting_for_human`, staff receives a notification, and AI stops according to policy.

Scenario E — Unknown question: The knowledge base has no reliable answer. AI should say it cannot confidently answer and offer human help rather than inventing a response.

37. Future Roadmap

- WhatsApp/SMS/Instagram and additional channel adapters.
- Advanced CRM synchronization.
- E-commerce product/catalog tools.
- More specialized agents.
- Voice agent with strict usage controls.
- Agent performance evaluation and A/B testing.
- Workflow automation builder.
- Advanced reporting and revenue attribution.
- Usage-based billing.
- Organization-level AI model selection.
- Local/private AI deployment options.
- Human-agent collaboration and suggested replies.

38. Final Implementation Checklist

- ☐ Finalize PRD and MVP boundaries.
- ☐ Create Git repositories and branching strategy.
- ☐ Set up Docker development environment.
- ☐ Create Next.js frontend.
- ☐ Create Laravel backend.
- ☐ Configure MySQL migrations.
- ☐ Configure Redis and queues.
- ☐ Configure Qdrant.
- ☐ Implement authentication.
- ☐ Implement organizations and RBAC.
- ☐ Implement tenant middleware/policies.
- ☐ Build conversation and message models.
- ☐ Build web chat widget.
- ☐ Build AI provider abstraction.
- ☐ Build orchestrator and structured tool layer.
- ☐ Implement usage/token logging.
- ☐ Implement knowledge upload and RAG pipeline.
- ☐ Implement Support Agent.
- ☐ Implement Sales Agent.
- ☐ Implement Appointment Agent.

- ☐ Implement lead/customer management.
- ☐ Implement calendar integration.
- ☐ Implement human handoff.
- ☐ Implement notifications.
- ☐ Implement analytics.
- ☐ Implement audit logging.
- ☐ Add security controls.
- ☐ Write unit/API/integration/E2E tests.
- ☐ Configure monitoring and backups.
- ☐ Deploy staging.
- ☐ Run acceptance tests.
- ☐ Deploy production.
- ☐ Document operations and developer onboarding.

Final recommendation

Build the system as a modular monolith first: Next.js frontend, Laravel backend, MySQL for business data, Redis for queues/cache, Qdrant for RAG, and a provider abstraction for AI. Keep Support, Sales, and Appointment agents modular, but let them share one conversation engine and one controlled tool layer. This gives the project a clean path from MVP to a larger AI automation platform without prematurely creating a complex microservice architecture.

End of PRD • Version 1.0